2026.10.23 · KO

# GovBiz

## 정부지원사업 탐색부터 신청 준비까지 연결하는 AI 서비스

SEARCH & RAG / ARCHITECTURE / DEPLOYMENT

### 검색

Elasticsearch + Qdrant + RRF

### 검증

AI 응답 검증 + 원문·인용 대조

### 구조

Spring Boot + FastAPI

### 배포

Vercel + AWS

# 목차

01 프로젝트 개요 03

02 시스템 아키텍처 04

03 맞춤 공고 검색 05 - 10

04 공고 상세 질문 RAG 11 - 13

05 협업 모집글 AI 에이전트 14 - 16

06 서비스 구조 17 - 18

07 애플리케이션 구조 19 - 22

08 배포와 복구 23 - 25

09 네트워크 구성 26

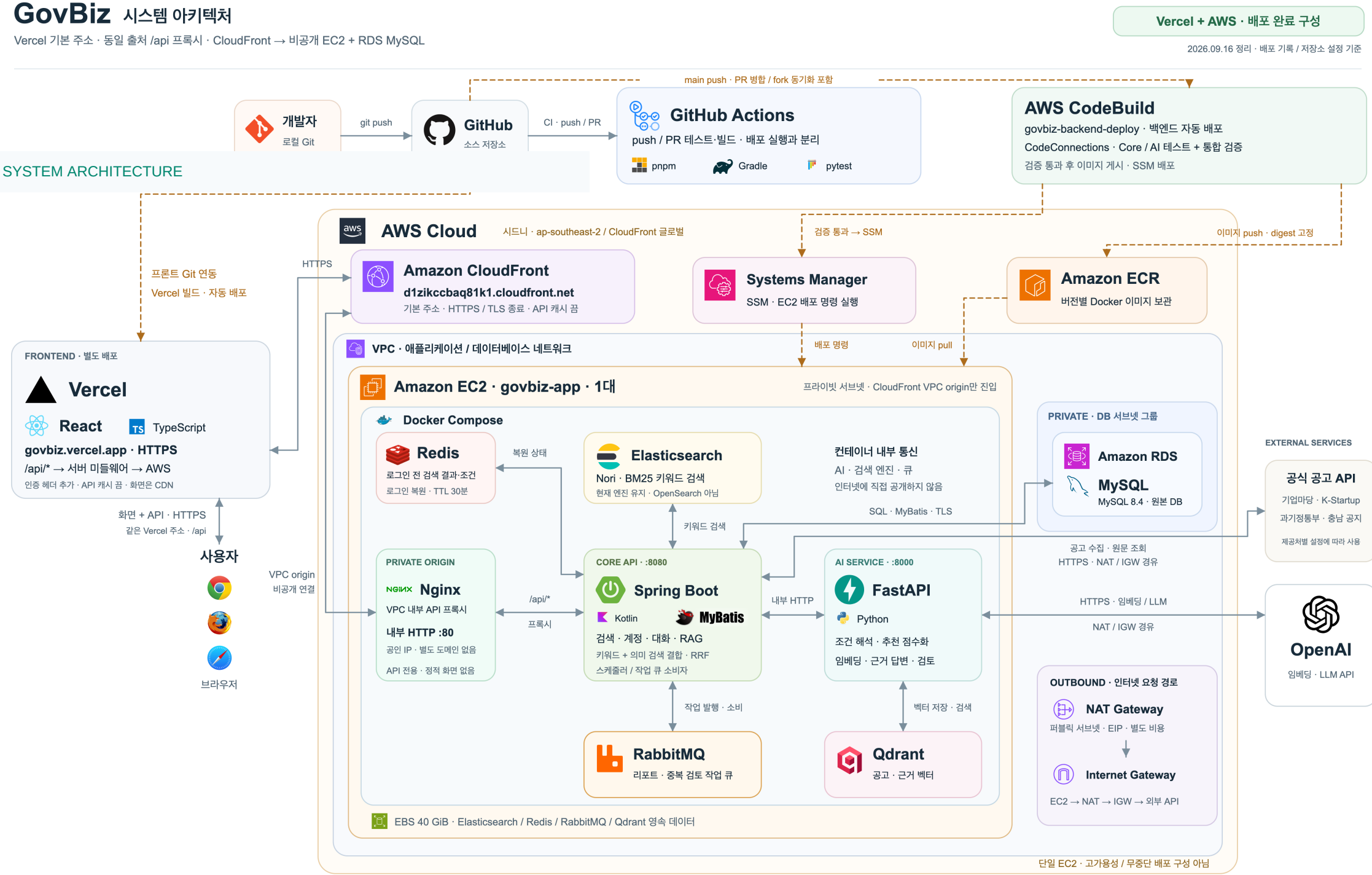
10 한국어 검색 트러블슈팅 27 - 28

# 공고 탐색을 신청 준비·진행 관리까지 연결합니다

여러 공식 제공처의 공고를 찾고 근거를 확인한 뒤, 관심함을 중심으로 문서 작성·일정·협업을 이어가는 서비스입니다.



# 시스템 아키텍처




실선: 서비스 통신 · 주황 점선: 배포 경로 · CodeBuild는 Core / AI 이미지만 교체 · 운영 환경값·Compose는 별도 관리

기존 배포 확인 기록 + 저장소 설정 기준 · 작성 시 AWS 세션 만료로 실시간 상태 재조회 안 됨 · 단일 EC2 / 중단 가능 · OAuth·백업 복구·부하 검증 완료를 뜻하지 않음 · 비밀값 미표기

# 한국어 표현을 분석하고 검색 범위를 넓힙니다

Elasticsearch에 Nori 분석 설정과 사전을 적용해 지명·복합어·유사 표현을 다룹니다.

01

## Nori 분해 모드

### discard 모드

복합어가 분해되면
분해된 단어를 색인에 사용

02

## 사용자 사전

### 지명·업무 용어를 직접 지정

횡성은 유지하고
대출이자는 대출·이자로 분석

03

## 검색 시점 동의어

### 질의에 유사 표현을 함께 고려

여행사 → 여행업체
융자와 보조금은 구분

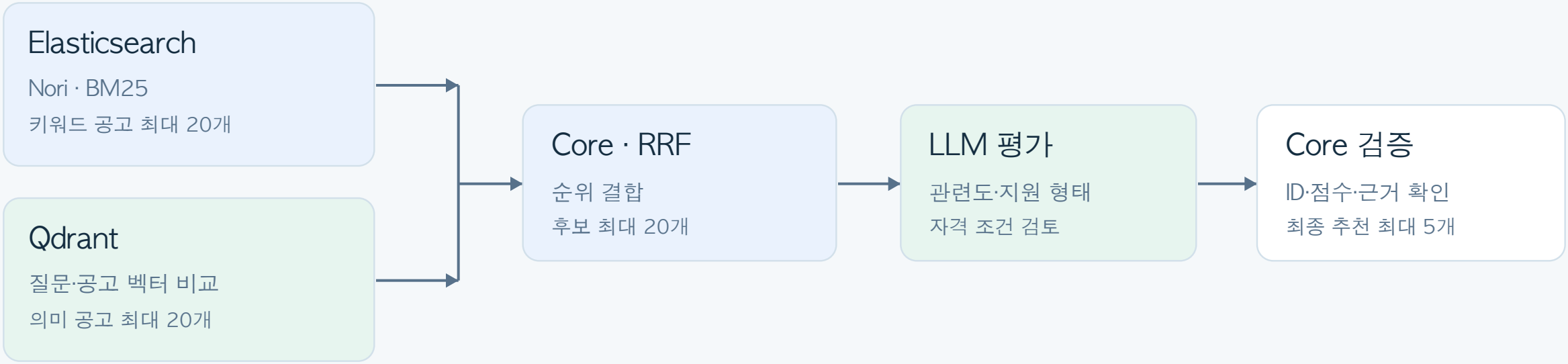
검색 범위는 넓히되, 지원 방식처럼 의미가 달라지는 표현은 묶지 않습니다.

Elasticsearch 분석 설정에 사용자 사전과 검색 시점 동의어를 추가했습니다.

# 공고 후보를 검색하고 추천 순서를 구성합니다

Core가 공개 중인 버전을 기준으로 후보를 제한하고, 두 검색 결과를 RRF로 결합합니다.

MySQL 공개 공고 — 공고 ID + 내용 해시 목록을 검색 대상에 전달




한계: 공개 대상 1만 건이면 식별 정보도 1만 건이어서 조회·전송·필터 비용이 늘 수 있습니다.

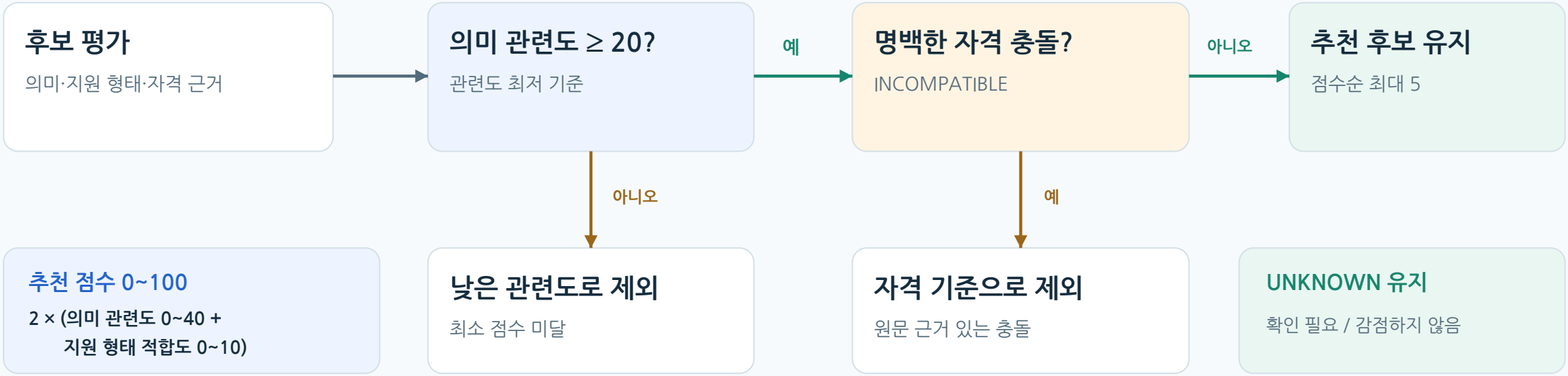
# 두 검색에 함께 오른 후보가 어떻게 합쳐지는지 봅니다

설명용 순위를 직접 계산한 예시입니다. 실제 공고의 검색 결과나 성능 측정치는 아닙니다.



# 관련도 기준과 자격 기준을 서로 다른 분기로 적용합니다

자격을 확인하지 못한 후보는 관련도에서 감점하지 않습니다. 그림은 선별 규칙의 요약이며 호출 순서도는 아닙니다.




추천 점수 100점은 선정 확률이 아닙니다. 자격 정보가 부족하면 UNKNOWN으로 유지해 추가 확인이 필요함을 표시합니다.

# 검색 준비를 마친 최신 수집 결과만 반영합니다

검색 데이터를 모두 준비한 뒤, 같은 제공처에서 가장 최근에 시작한 작업인지 확인합니다.




**검색 결과 확인** 현재 MySQL에서 제공하는 공고와 내용이 같은 검색 결과만 사용합니다.

### 검색 데이터 준비 순서

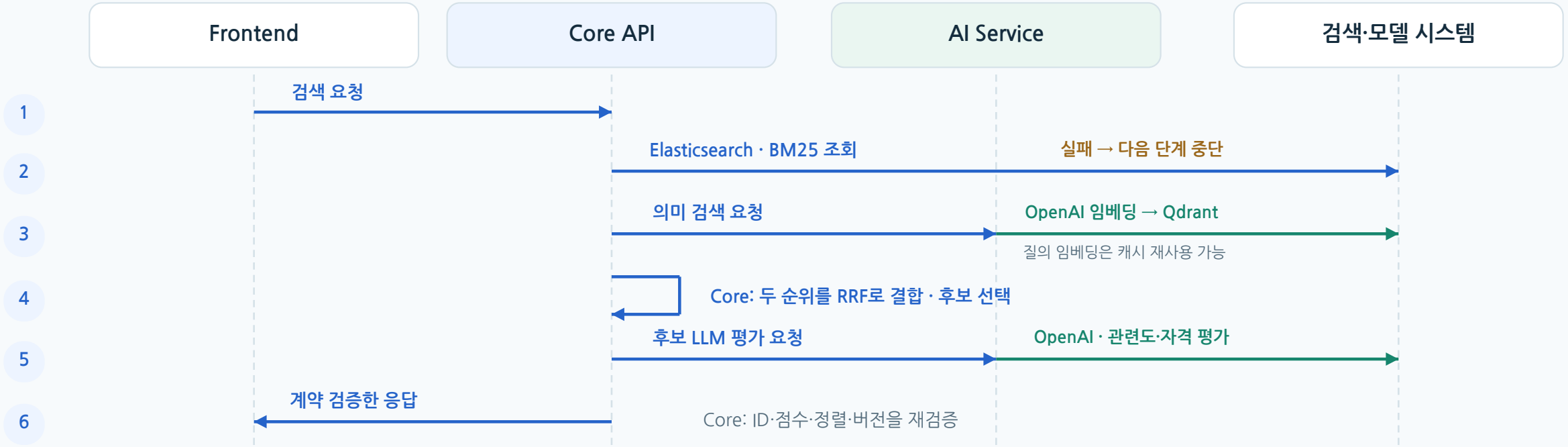
Elasticsearch 준비 후 Qdrant를 준비합니다.
40건이면 16·16·8건으로 요청하며, 공고마다 벡터를 따로 준비합니다.

### 작업 번호 비교 예시

1번 작업 뒤에 2번이 시작됐다면 1번 결과는 반영하지 않습니다.
2번이 실패해도 마지막으로 정상 반영한 공고를 유지합니다.

# 검색 호출 순서로 비용과 장애 경계를 통제합니다

Core가 검색 순서를 조율합니다. Elasticsearch 성공 뒤에 의미 검색을 호출하고, 후보를 합친 뒤 LLM으로 평가합니다.



# 선택한 공고의 원문에서 답변 근거를 찾습니다

Core와 AI Service가 원문을 준비하고, 질문에 필요한 구간을 확인해 답변합니다.

**예시 질문** “신청 기간이 언제인가요?” → 신청 기간이 적힌 원문 구간을 찾아 답변에 활용

## 01-03 원문과 검색 데이터를 준비합니다

01 CORE

### 공식 원문 확보

선택한 기업마당 공고의
원문을 가져오거나 재사용

→

02 CORE

### 원문을 구간으로 나누기

구간당 최대 1,500자,
공고당 최대 50개 구간

→

03 AI SERVICE

### 구간별 벡터 준비

구간을 벡터로 만들어 Qdrant에 저장
이미 같은 벡터가 있으면 재사용

## 04-06 질문에 맞는 근거를 찾고 답변합니다

04 CORE → AI SERVICE

### 질문과 검색 범위 전달

질문과 현재 공고의 구간 목록 전달
Qdrant에서 관련 구간 최대 5개 검색

→

05 CORE

### 검색 결과 확인

돌아온 구간 ID·해시·순서를 대조
Core가 가진 원문 구간을 복원

→

06 AI SERVICE → CORE

### 답변과 출처 확인

AI가 검증된 원문으로 답변 작성
Core가 인용을 확인해 출처 제공

인용한 구간이 실제 전달된 원문에 속하는지 확인합니다. 답변 전체의 의미적 정확성까지 보장하지는 않습니다.

# 검색 결과와 답변의 인용을 두 번 확인합니다

Core가 검색 직후 원문 구간을 확인하고, 답변 생성 후 인용 대상을 다시 확인합니다.

**예시** “신청 기간이 언제인가요?” → 선택한 공고 A의 구간 2를 찾았다고 가정

## 01 검색 직후 — 찾은 구간이 선택한 공고의 원문인지 확인

### 1 Core가 구간 준비

공고 A의 원문을 구간으로 나눔

**구간 2 · ID x2 · 해시 abc**

→

### 2 AI가 관련 구간 검색

질문과 관련된 구간을 찾음

**구간 2 · ID x2 · 해시 abc 반환**

→

### 3 Core가 결과 대조

준비한 구간 정보와 같은지 확인

일치 → Core의 원문 사용

불일치 → 오류 처리

## 02 답변 생성·인용 확인 — 전달한 구간을 인용했는지 확인

### 1 Core가 원문 전달

확인한 원문 구간을 AI에 전달

→

### 2 AI가 답변 생성

전달받은 원문을 근거로 답변

→

### 3 AI가 인용 ID 전달

답변과 인용 구간 ID 반환

→

### 4 Core가 인용 확인

전달된 구간인지 확인

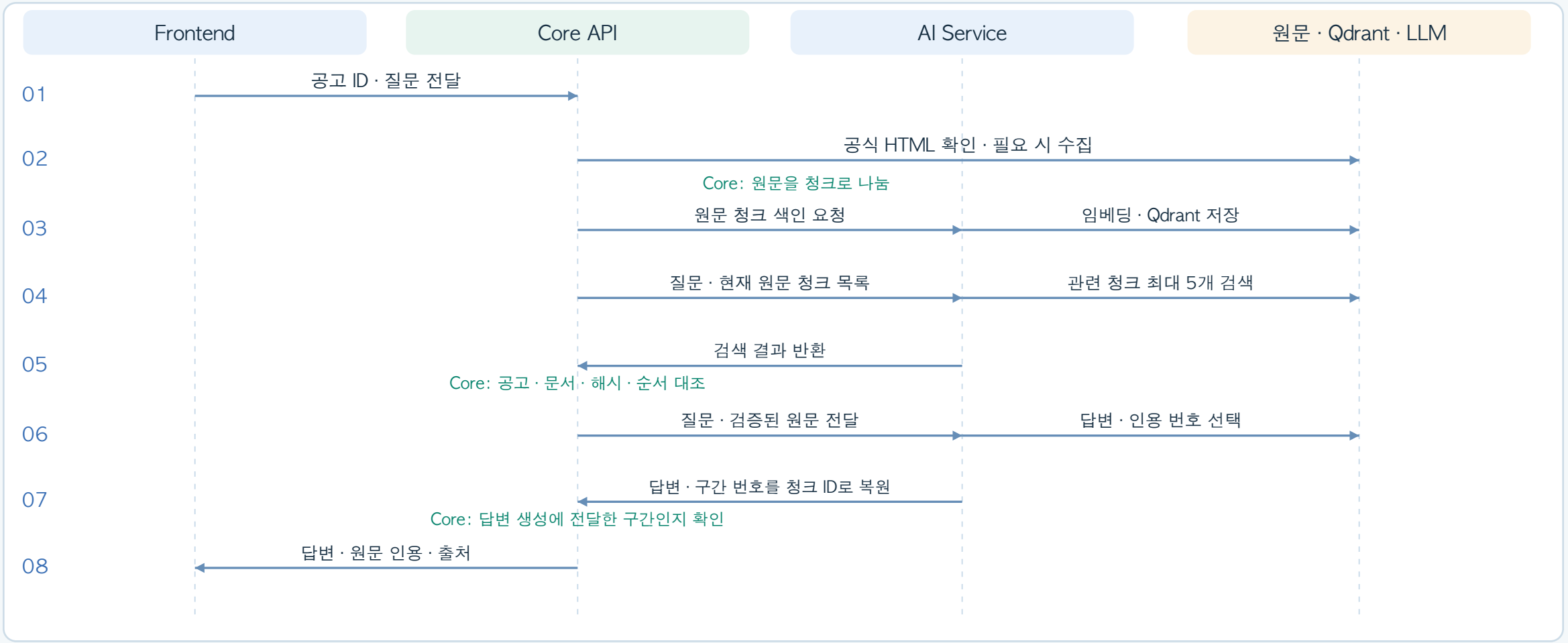
확인한 원문과 출처 제공

구간 ID·해시는 예시입니다. 근거가 부족하면 확인이 필요하다고 표시합니다.

출처 확인만으로 답변 전체의 의미적 정확성을 보장하지는 않습니다.

# 공고 상세 질문의 원문 RAG 처리 흐름

원문 준비부터 관련 구간 검색, 답변과 인용 검증까지의 서비스 간 흐름입니다.




원문 구간의 존재와 출처를 확인하며, 답변 전체의 의미적 정확성을 보장하지는 않습니다.

# 대화와 회원 상태를 함께 전달합니다

협업 모집글을 찾는 자유 질문은 Context를 통해 앞선 대화와 이어집니다.

질문 예시 “서울에서 우리 기업에 맞는 곳을 찾아줘”

01 FRONTEND
### 현재 질문과 대화
현재 질문 + 최근 대화 최대 6개
화면 상태와 도움말을 함께 전달

→

02 CORE API
### 개인정보와 회원 확인
질문·대화의 개인정보 마스킹
로그인·기업 등록 상태 확인

→

03 CLASSIFY
### 질문 의도 분류
질문과 Context를 LLM에 전달
협업 모집글 질문으로 분류

→

04 LANGGRAPH
### 회원 도구 경로
PARTNER_MATCH → plan
기업 정보는 이후 도구로 조회

## 대화 Context

### 현재 질문 · 대화 내역
“서울”은 이번 질문에서 전달

### 화면 · 도움말
현재 화면과 이용 안내를 참고

### 회원 상태
인증 여부와 기업 등록 상태

실행 조건: 자유 질문 AI·회원 도구 에이전트 활성화, 로그인과 기업 등록 필요 (기본 비활성화)

# 필요한 정보를 조회하며 도구 호출을 반복합니다

LLM이 도구를 고르고, 실행 결과를 보고 추가 호출 여부를 판단합니다.

도구 예시

get_my_company_profile() · search_partner_recruitments(region="서울")

01 PLAN / LLM

## 도구와 인자 선택

질문·대화·이전 도구 결과 전달
bind_tools로 도구와 입력 형식 제공

02 TOOLS

## 선택한 도구 실행

Core의 읽기 전용 API 호출
기업 정보 또는 모집글 조회

03 STATE

## 실행 결과 기록

messages · tool_results에 기록
성공 여부와 호출 횟수 확인

모든 도구 성공 + 호출 여유 → plan으로 돌아가 재판단

### 계속 조회

추가 도구가 필요하면 다시 실행합니다.

### 답변으로 이동

실행할 도구 없음 · 실패 · 호출 상한

동일 도구·인자 중복 차단 · 도구 호출 기본 최대 3회 · 선택과 순서는 모델이 결정

# 도구 결과를 기억하고, 답변의 카드를 검증합니다

상태는 한 요청에서만 유지하며, 결과에 없는 카드가 전달되지 않도록 확인합니다.

## 01 요청 단위 Memory

LANGGRAPH STATE

### messages

AIMessage와 ToolMessage를 누적
다음 plan이 앞선 도구 호출과 결과를 참고

LANGGRAPH STATE

### tool_results

도구 이름·인자·성공 여부·결과 저장
중복 호출 방지와 답변·카드 검증에 활용

요청마다 상태를 새로 만들며 체크포인터를 통한 장기 기억은 사용하지 않습니다.

## 02 답변 생성과 검증

ANSWER / LLM

### 답변과 카드 ID 선택

성공한 도구 결과를 바탕으로 생성

→

VERIFY / AI SERVICE

### 조회 결과와 ID 대조

잘못된 카드가 있으면 한 번 재생성

→

FINALIZE / AI SERVICE

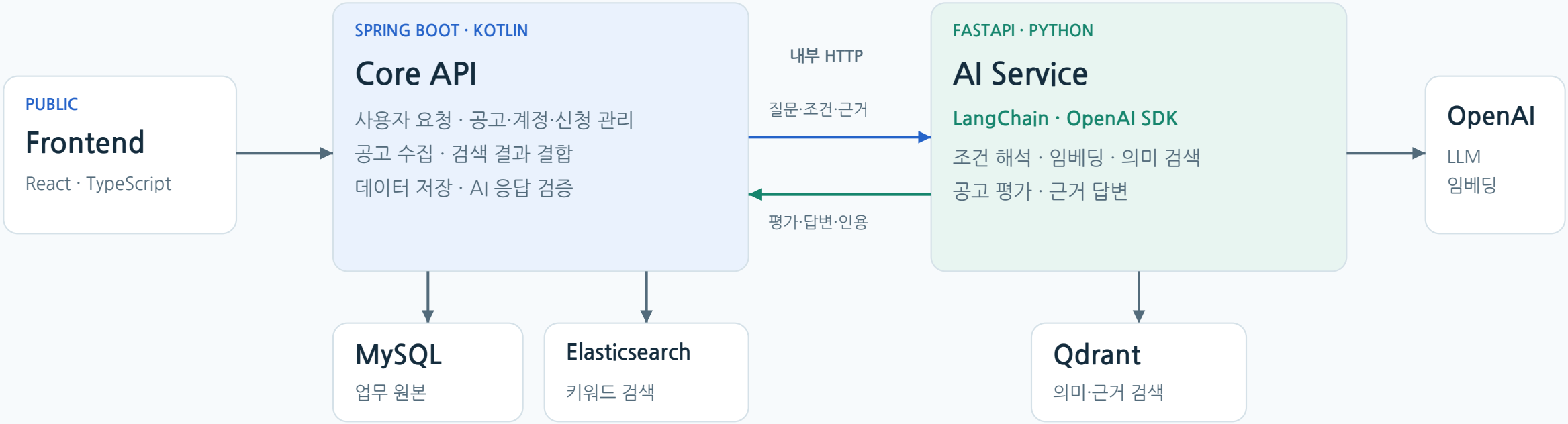
### 카드 결과 정리

재검증 실패 시 카드 전체 제외

Core는 응답 형식과 허용 경로를 확인합니다. 카드 존재 확인이 추천 이유의 의미적 정확성을 보장하지는 않습니다.

# 서비스 구조 — 화면·업무·AI 실행의 역할을 나눕니다

Python의 LangChain·OpenAI SDK를 활용하고, 업무 데이터 처리와 AI 실행의 책임을 분리했습니다.




React는 **화면**, Core API는 **업무 처리·검색 결합·응답 검증**, AI Service는 **의미 검색·LLM 실행**을 담당합니다.

# 데이터 구조 — 원본·검색 색인·임시 상태·작업 전달

공고 원본과 업무 기록은 Core API가 관리합니다. AI Service는 MySQL에 직접 접근하지 않고 전달받은 데이터로 검색·생성을 실행합니다.




MySQL을 **원본**으로 두고, Elasticsearch·Qdrant는 **검색 색인**, Redis는 **임시 검색 결과 보관**, RabbitMQ는 **비동기 작업 전달**에 사용합니다.

# 요청 접수·외부 통신·DB 접근의 역할을 나눴습니다

지원사업 기능(supportprogram) 안에서 역할을 나누고, Service가 업무에 필요한 경로를 선택합니다.

**Controller**
HTTP 요청을 받고 응답

→

**Service**
조회·호출 순서와 업무 흐름 조율

**필요한 업무에 따라 아래 경로를 사용**
외부 정보 요청 또는 저장된 데이터 조회·저장

## 외부 정보가 필요할 때

단순 호출은 Facade 없이 Client로

**Service**
외부 정보 요청

→

**Facade · 필요한 경우만**
복잡한 응답 검증·변환

→

**Client**
외부 서버와 HTTP 통신

→

**외부 API**
AI Service·기업마당

## 저장된 공고를 읽거나 쓸 때

**Service**
조회·저장 요청

→

**Repository**
조회·저장, 데이터 변환

→

**MyBatis Mapper**
SQL 실행 요청

→

**XML**
실행할 SQL

→

**MySQL**
공고 데이터

Repository 경계: 업무 모델 ↔ DbRow · DB 행 형식은 외부로 내보내지 않음

**Domain · 프레임워크와 무관한 업무 규칙**
예: 신청 기간과 오늘 날짜로 접수 상태 계산

**예: 공고 목록 보기 → DB 조회 경로 사용**
저장된 공고 조회 → 접수 상태 계산 → 필터·정렬 → 목록 응답

# 화면·업무·서버 통신의 역할을 나눴습니다

주요 업무 기능의 설계 기준과 대표 조회 흐름입니다. 모든 기능에 일관되게 적용하지는 못했습니다.

## 01 역할과 코드 의존 방향

### Presentation · 화면

React View + ViewModel
사용자 동작 · 입력 확인 · 결과 표시
입력 확인: 필수값·글자 수 등

### Domain · 업무

UseCase: 업무 흐름 처리
Repository 계약: 필요한 조회 기능의 약속
구체적인 HTTP 구현을 직접 알지 않음

### Data · 서버 통신

Repository 구현체 · HTTP 호출
Zod로 서버 응답의 형식 검증
DTO → 업무 모델 변환

코드 의존 **Presentation** → **Domain** ← **Data** Data가 Domain의 계약을 구현

## 02 실제 조회 흐름

예: 사용자가 지역 필터를 변경합니다.

**ViewModel**
조회 요청

→

**UseCase**
조건 정리·검사

→

**Repository 구현체**
계약에 맞춰 HTTP 호출

→

**Core API**
조회 결과 응답

**응답 처리** Data: Zod 검증 → DTO를 업무 모델로 변환 → UseCase에 반환 → ViewModel이 화면 상태 갱신

## 03 Awilix 연결

Data의 Repository 구현체 → Domain의 UseCase에 주입

앱 구성 단계에서 연결

# MVVM — ViewModel로 화면과 업무 호출 분리

공고 목록 화면의 예: 화면은 표시하고, ViewModel은 조회와 상태를 연결합니다.

Presentation

## View · 화면

목록 · 로딩 · 오류 표시

필터 UI · 이벤트 연결

조건·재시도 →

← 화면 상태

Presentation

## ViewModel

useEffect로 UseCase 실행

조건 변경 · 재시도 시 조회

useState로 조회 상태 관리

결과 · 로딩 · 오류 · 재시도 제공

업무 호출 →

← 조회 결과

Domain

## Model · 업무

BrowseSupportProgramsUseCase

입력 정리 · 검사

Repository 계약으로 데이터 조회

### ViewModel이 화면에 제공

조회 상태(phase) · 결과(data)

필터 옵션 · 재시도(retry)

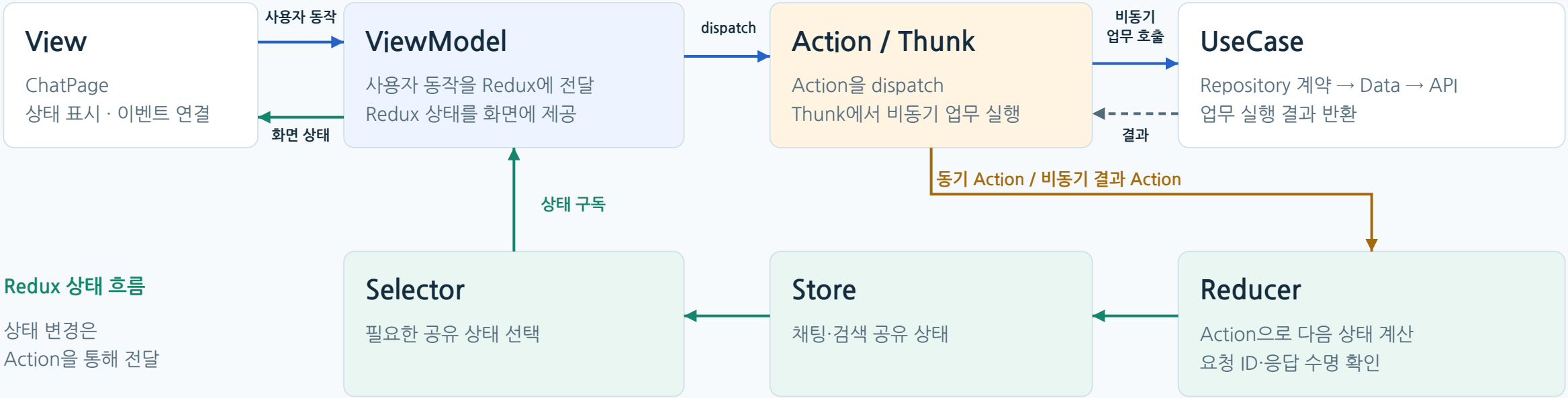
### 역할의 경계

View는 UseCase·Repository를 직접 실행하지 않습니다.

업무 규칙은 UseCase에 둡니다.

# Flux: Redux의 상태 흐름도 ViewModel을 통해 연결합니다

채팅 화면의 예: ViewModel을 거쳐 공유 상태를 한 방향으로 갱신합니다.




**Redux 상태 흐름**

상태 변경은
Action을 통해 전달

### ViewModel로 비즈니스로직 상태관리 분리

View는 ViewModel을 통해 동작을 요청하고 상태를 받습니다.
메시지·검색 등 공유 상태는 Redux에서 관리합니다.

### Reducer는 상태 계산만 담당

외부 API 호출은 Thunk가 UseCase에 위임합니다.
Reducer 안에서는 외부 API를 호출하지 않습니다.

# 코드를 검사하고 이미지를 EC2에 배포합니다

프론트엔드는 Vercel에서 배포하고, 백엔드는 CodeBuild가 아래 과정을 실행합니다.

**시작** main에 코드 반영 → CodeBuild 실행 · CodeConnections로 GitHub 소스 연결

01

## 검증 게이트

Core 빌드·테스트
AI 테스트·패키지 빌드
Compose·Nginx 등 통합 확인
실패하면 배포 중단

02

## 이미지 준비

Core·AI 실행 이미지 준비
커밋 ID를 이미지 태그로 지정
예: git-abc123…
같은 태그가 이미 있으면 재사용

03

## ECR 게시

이미지를 ECR에 저장
저장된 이미지의 digest 확인
digest로 이미지 주소를 고정

04

## SSM 배포

CodeBuild가 SSM에 명령 요청
지정 EC2에 이미지 주소 전달
EC2의 배포 스크립트 실행
서비스 교체·상태 확인 → 22쪽

### 커밋 태그와 digest

커밋 태그: 어떤 코드에서 만든 이미지인지 표시

digest: 실제 이미지 내용을 식별하는 고유값

### 오래된 배포 방지

ECR 게시 전과 후 최신 main 커밋인지 확인

이미 오래된 빌드는 EC2에 배포하지 않음

통합 검증의 OpenAI 호출은 스텁을 사용합니다. 테스트 통과가 실제 LLM 답변 품질을 보장하지는 않습니다.

# 배포 전 확인부터 상태 검증·복구까지

EC2 배포 스크립트가 확인과 교체를 순서대로 실행합니다.

01 사전 점검

## 배포 전에 확인

다른 배포가 동시에 실행되지 않도록 잠금

Docker 저장 공간 4 GiB 이상 확인

새 이미지의 저장소·커밋·실행 환경 확인

기존 서비스와 이미지 설정 상태 확인

02 서비스 교체

## AI → Core 순서

AI를 교체하고 정상 실행을 확인

Core를 교체해 정상 실행을 확인

Nginx 설정을 검사하고 다시 불러옴

03 상태 검증

## 교체 후 다시 확인

Core·AI·Nginx 등 서비스 상태 확인

Core에서 AI 상태 확인 API를 호출

AI 연결 결과가 정상(up)인지 확인

Qdrant는 실행 중인지 확인

### 교체·상태 확인 중 실패하면

배포 전 환경 파일을 복원하고 이전 이미지로 재실행 시도

복구에 성공해도 이번 배포는 실패로 기록

### 유지하는 대상

검색 엔진·캐시·큐 컨테이너와 영속 데이터 유지

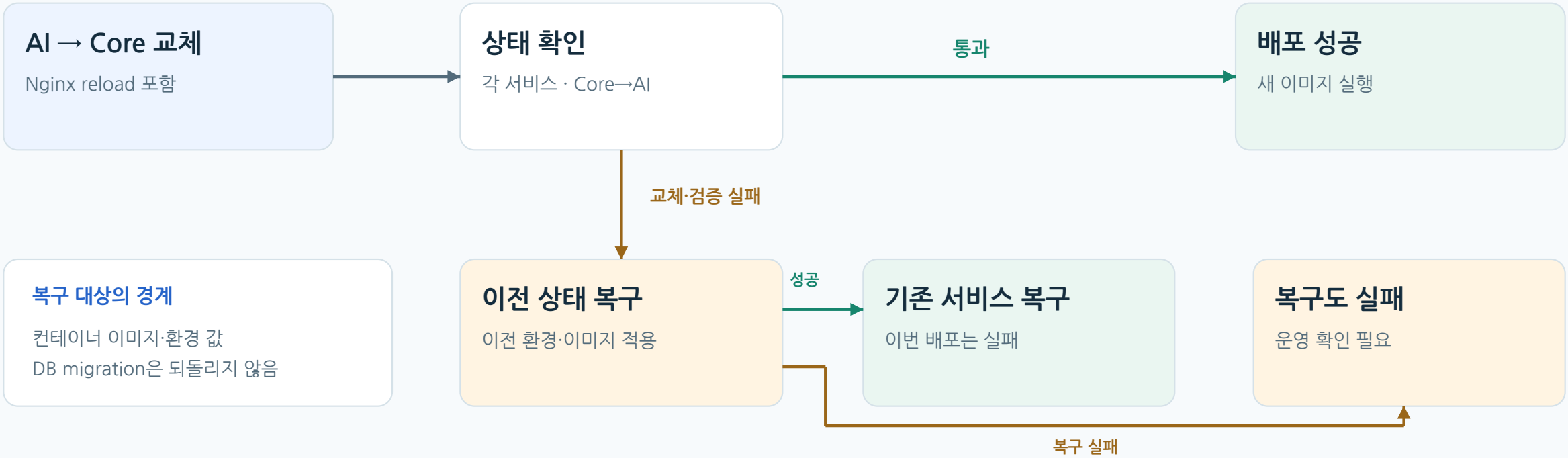
운영 환경·Compose 전체를 자동으로 덮어쓰지 않음

Nginx 설정·상태 확인은 외부 사용자 요청의 성공까지 보장하지 않습니다.

서버 한 대를 교체하는 동안 서비스가 중단될 수 있으며, 이미지 복구로 DB 변경을 되돌리지는 않습니다.

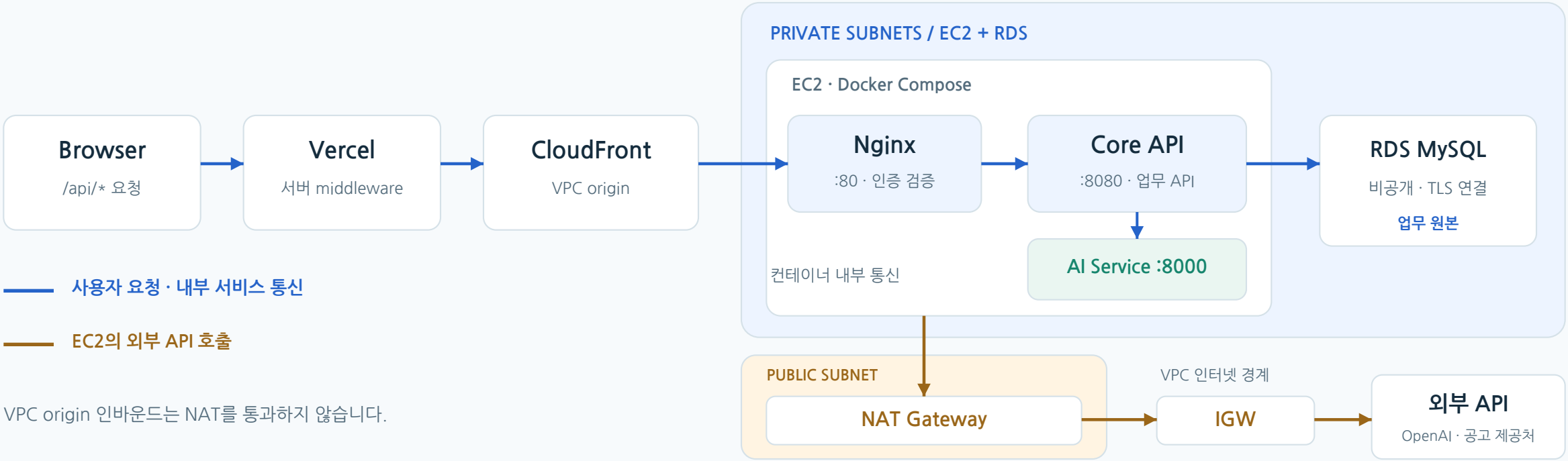
# 교체 실패와 복구 실패를 별도 결과로 남깁니다

배포는 상태 확인을 거쳐 종료합니다. 이전 이미지 복구에 성공해도 이번 배포가 성공한 것으로 바꾸지 않습니다.



# 운영 구조 — Vercel과 비공개 AWS를 연결합니다

EC2와 RDS는 프라이빗 서브넷에 둡니다. 사용자 API 요청은 Vercel과 CloudFront를 거치고, 서버의 외부 호출은 NAT Gateway를 사용합니다.




**백엔드 배포** CodeBuild 검증·빌드 → ECR 이미지 저장 → SSM 배포 명령 → EC2 서비스 교체

프론트엔드는 **Vercel**에서 제공하고, EC2·RDS는 **프라이빗 네트워크**에 배치했습니다. 백엔드는 **CodeBuild·ECR·SSM**을 통해 배포합니다.

# 실제 목표 공고로 검색 누락을 살펴봤습니다

질문에 맞춰 미리 선정한 공고가 키워드 검색 결과에 나오는지 비교했습니다.

목표 공고

〔강원〕 횡성군 2026년 중소기업육성자금 이차보전 지원 공고

질문 “횡성 소상공인의 대출이자를 지원하는 공고 찾아줘”

### 문제 · 변경 전

목표 공고가 상위 20개 밖

지역·복합어 분석 차이를 점검

### 원인 분석 · 조치

횡성 분석과 사용자 사전 점검

복합어 분해 모드 함께 보완

### 변경 후

목표 공고가 1위에 표시

고정 질문에 대한 비교 결과

설정을 함께 바꿨으므로 개선 효과를 특정 설정 하나의 결과로 단정할 수 없습니다.

# 한국어 검색에서 누락되던 공고를 줄였습니다

고정 데이터와 질문을 사용해 Elasticsearch 분석 설정 변경 전후를 비교했습니다.

### 목표 공고 Hit@20

295 / 300 → 299 / 300

기존 누락 5개 중 4개 회복 · 1개 미적중

### 비교 조건

공고 1,422건 · 개발 중 AI 작성 질문 300개

질문별로 미리 정한 목표 공고를 찾는지 비교

설정·사용자 사전·동의어 변경 전후

### 결과의 해석 범위

키워드 검색의 목표 공고 적중 여부이며, 전체 관련 공고 Recall이나
최종 하이브리드 추천 정확도를 측정한 수치는 아닙니다.

남은 누락 1개와 순위가 하락한 질문 9개도 함께 기록했습니다.